

Ian Keith
Ruohong Zhu
Vishvaadarshan Ashoknathan
DSC 623
Project:Part 3

A. Code

```
import sqlite3

conn = sqlite3.connect("rental_company.db")

cursor = conn.cursor()

cursor.execute("PRAGMA foreign_keys = ON;")

print("Database connected successfully.")


# Create Outlet table
cursor.execute("""
CREATE TABLE IF NOT EXISTS Outlet (
    Outlet_number INTEGER PRIMARY KEY,
    Address TEXT NOT NULL,
    Phone_number TEXT NOT NULL,
    Fax_number TEXT
);
""")

# print("Outlet table created successfully.")


# Create Vehicle table
cursor.execute("""
CREATE TABLE IF NOT EXISTS Vehicle (
    Registration_number TEXT PRIMARY KEY,
    Make TEXT NOT NULL,
    Model TEXT NOT NULL,
    Engine_size REAL NOT NULL CHECK (Engine_size > 0),
    Capacity INTEGER NOT NULL CHECK (Capacity > 0),
    Current_mileage INTEGER NOT NULL CHECK (Current_mileage >= 0),
    Daily_hire_rate REAL NOT NULL CHECK (Daily_hire_rate > 0),
    Outlet_number INTEGER NOT NULL,
```

```
        FOREIGN KEY (Outlet_number) REFERENCES Outlet(Outlet_number)
    );
    """
```

```
#print("Vehicle table created successfully.")
```

```
# Create Client table
cursor.execute("""
CREATE TABLE IF NOT EXISTS Client (
    Client_number INTEGER PRIMARY KEY,
    First_name TEXT NOT NULL,
    Last_name TEXT NOT NULL,
    Address TEXT NOT NULL,
    Phone_number TEXT NOT NULL,
    DOB TEXT NOT NULL,
    Driver_license_number TEXT NOT NULL UNIQUE
);
""")
```

```
# print("Client table created successfully.")
```

```
# Create Staff table
cursor.execute("""
CREATE TABLE IF NOT EXISTS Staff (
    Staff_number INTEGER PRIMARY KEY,
    First_name TEXT NOT NULL,
    Last_name TEXT NOT NULL,
    Home_address TEXT NOT NULL,
    Home_phone_number TEXT NOT NULL,
    DOB TEXT NOT NULL,
    Sex TEXT NOT NULL CHECK (Sex IN ('M', 'F', 'Other')),
    Date_joined TEXT NOT NULL,
    Job_title TEXT NOT NULL,
    Salary REAL NOT NULL CHECK (Salary > 0),
    Outlet_number INTEGER NOT NULL,
    FOREIGN KEY (Outlet_number) REFERENCES Outlet(Outlet_number)
);
""")
```

```
# print("Staff table created successfully.")
```

```
# Create Hire_Agreement table
cursor.execute("""
CREATE TABLE IF NOT EXISTS Hire_Agreement (
```

```

Hire_number INTEGER PRIMARY KEY,
Date_started TEXT NOT NULL,
Date_wish_end TEXT NOT NULL,
Mileage_before INTEGER NOT NULL CHECK (Mileage_before >= 0),
Mileage_after INTEGER,
Client_number INTEGER NOT NULL,
Registration_number TEXT NOT NULL,

FOREIGN KEY (Client_number) REFERENCES Client(Client_number),
FOREIGN KEY (Registration_number) REFERENCES Vehicle(Registration_number),

```

```

CHECK (Mileage_after IS NULL OR Mileage_after >= Mileage_before)

```

```

);
"""

```

```

# print("Hire_Agreement table created successfully.")

```

```

# Insert data into Outlet
cursor.executemany("""
INSERT OR IGNORE INTO Outlet VALUES (?, ?, ?, ?);
""", [
    (1, "100 Miami Ave, Miami, FL", "305-111-1001", "305-111-2001"),
    (2, "200 Coral Way, Coral Gables, FL", "305-222-1002", "305-222-2002"),
    (3, "300 Ocean Dr, Miami Beach, FL", "305-333-1003", None),
    (4, "400 Brickell Ave, Miami, FL", "305-444-1004", "305-444-2004"),
    (5, "500 Kendall Dr, Kendall, FL", "305-555-1005", None)
])

```

```

# Insert data into Vehicle
cursor.executemany("""
INSERT OR IGNORE INTO Vehicle VALUES (?, ?, ?, ?, ?, ?, ?, ?);
""", [
    ("ABC123", "Honda", "Civic", 1.8, 5, 25000, 45.00, 1),
    ("DEF456", "Toyota", "Camry", 2.5, 5, 32000, 55.00, 1),
    ("GHI789", "Honda", "Accord", 2.0, 5, 41000, 60.00, 2),
    ("JKL012", "Ford", "Explorer", 3.0, 7, 50000, 80.00, 3),
    ("MNO345", "BMW", "X5", 3.0, 5, 28000, 120.00, 4)
])

```

```

# Insert data into Client
cursor.executemany("""
INSERT OR IGNORE INTO Client VALUES (?, ?, ?, ?, ?, ?, ?);
""", [
    (1, "John", "Smith", "10 Pine St, Miami, FL", "305-101-0001", "1990-05-12", "DL10001"),

```

```

(2, "Maria", "Garcia", "20 Oak St, Miami, FL", "305-102-0002", "1988-09-23", "DL10002"),
(3, "David", "Lee", "30 Palm St, Miami, FL", "305-103-0003", "1995-01-18", "DL10003"),
(4, "Sarah", "Brown", "40 Bay St, Miami, FL", "305-104-0004", "1985-11-04", "DL10004"),
(5, "Michael", "Chen", "50 Sunset St, Miami, FL", "305-105-0005", "1992-07-30", "DL10005")
])

```

```

# Insert data into Staff

```

```

cursor.executemany("""
INSERT OR IGNORE INTO Staff VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?);
""", [
    (1, "Alice", "Johnson", "101 Staff Rd, Miami, FL", "305-201-0001", "1980-03-14", "F",
"2020-01-10", "Manager", 65000, 1),
    (2, "Robert", "Wilson", "102 Staff Rd, Miami, FL", "305-202-0002", "1979-08-20", "M",
"2021-06-15", "Sales Assistant", 42000, 1),
    (3, "Emily", "Davis", "103 Staff Rd, Miami, FL", "305-203-0003", "1991-12-01", "F",
"2022-03-22", "Clerk", 38000, 2),
    (4, "James", "Miller", "104 Staff Rd, Miami, FL", "305-204-0004", "1987-04-09", "M",
"2019-09-01", "Mechanic", 50000, 3),
    (5, "Olivia", "Martinez", "105 Staff Rd, Miami, FL", "305-205-0005", "1993-10-17", "Other",
"2023-02-11", "Receptionist", 36000, 4)
])

```

```

# Insert data into Hire_Agreement

```

```

cursor.executemany("""
INSERT OR IGNORE INTO Hire_Agreement VALUES (?, ?, ?, ?, ?, ?, ?);
""", [
    (1, "2025-01-15", "2025-01-20", 25000, 25200, 1, "ABC123"),
    (2, "2025-03-10", "2025-03-15", 32000, 32500, 2, "DEF456"),
    (3, "2025-06-05", "2025-06-12", 41000, 41800, 1, "GHI789"),
    (4, "2025-09-18", "2025-09-25", 50000, 50750, 3, "JKL012"),
    (5, "2024-12-01", "2024-12-05", 28000, 28300, 4, "MNO345")
])

```

```

# print("Data inserted successfully.")

```

```

print("\nQuery 1: Vehicles hired by Client 1")

```

```

cursor.execute("""
SELECT

```

```

        Client.Client_number,
        Client.First_name,
        Client.Last_name,
        Vehicle.Registration_number,
        Vehicle.Make,
        Vehicle.Model
FROM Client
JOIN Hire_Agreement
    ON Client.Client_number = Hire_Agreement.Client_number
JOIN Vehicle
    ON Hire_Agreement.Registration_number = Vehicle.Registration_number
WHERE Client.Client_number = 1;
""")

```

```

for row in cursor.fetchall():
    print(row)

```

```

print("\nQuery 2: Vehicles above 30,000 miles hired during 2025")

```

```

cursor.execute("""
SELECT
    Vehicle.Registration_number,
    Vehicle.Make,
    Vehicle.Model,
    Vehicle.Current_mileage,
    Hire_Agreement.Date_started,
    Client.First_name,
    Client.Last_name
FROM Vehicle
JOIN Hire_Agreement
    ON Vehicle.Registration_number = Hire_Agreement.Registration_number
JOIN Client
    ON Hire_Agreement.Client_number = Client.Client_number
WHERE Vehicle.Current_mileage > 30000
    AND Hire_Agreement.Date_started BETWEEN '2025-01-01' AND '2025-12-31';
""")

```

```

for row in cursor.fetchall():
    print(row)

```

```
print("\nQuery 3: Total number of Honda vehicles across all outlets")
```

```
cursor.execute("""
SELECT
    Make,
    COUNT(*) AS Total_Hondas
FROM Vehicle
WHERE Make = 'Honda';
""")
```

```
for row in cursor.fetchall():
    print(row)
```

```
print("\nQuery 4: Vehicle make with the highest daily hire rate")
```

```
cursor.execute("""
SELECT Make, Daily_hire_rate
FROM Vehicle
WHERE Daily_hire_rate = (
    SELECT MAX(Daily_hire_rate)
    FROM Vehicle
);
""")
```

```
for row in cursor.fetchall():
    print(row)
```

```
print("\nQuery 5: Top 10 outlets with the most staff")
```

```
cursor.execute("""
SELECT
    Outlet_number,
    COUNT(*) AS Staff_Count
FROM Staff
GROUP BY Outlet_number
ORDER BY Staff_Count DESC
LIMIT 10;
```

```
""")
```

```
for row in cursor.fetchall():  
    print(row)
```

```
conn.commit()  
conn.close()
```

B. TABLE OUTPUTS

Client Table

```
# Create Client table  
cursor.execute("""  
CREATE TABLE IF NOT EXISTS Client (  
    Client_number INTEGER PRIMARY KEY,  
    First_name TEXT NOT NULL,  
    Last_name TEXT NOT NULL,  
    Address TEXT NOT NULL,  
    Phone_number TEXT NOT NULL,  
    DOB TEXT NOT NULL,  
    Driver_license_number TEXT NOT NULL UNIQUE  
) ;  
""")  
  
# print("Client table created successfully.")
```

Client_number	First_name	Last_name	Address	Phone_number	DOB	Driver_license_number
1	John	Smith	10 Pine St, Miami, FL	305-101-0001	1990-05-12	DL10001
2	Maria	Garcia	20 Oak St, Miami, FL	305-102-0002	1988-09-23	DL10002
3	David	Lee	30 Palm St, Miami, FL	305-103-0003	1995-01-18	DL10003
4	Sarah	Brown	40 Bay St, Miami, FL	305-104-0004	1985-11-04	DL10004
5	Michael	Chen	50 Sunset St, Miami, FL	305-105-0005	1992-07-30	DL10005

This code sets Client_Number as the primary key, lists the required attributes and creates the Client Table. This code also sets a rule that the driver_license_number allows for unique values.

Hire_agreement Table

```
# Create Hire_Agreement table
cursor.execute("""
CREATE TABLE IF NOT EXISTS Hire_Agreement (
    Hire_number INTEGER PRIMARY KEY,
    Date_started TEXT NOT NULL,
    Date_wish_end TEXT NOT NULL,
    Mileage_before INTEGER NOT NULL CHECK (Mileage_before >= 0),
    Mileage_after INTEGER,
    Client_number INTEGER NOT NULL,
    Registration_number TEXT NOT NULL,

    FOREIGN KEY (Client_number) REFERENCES Client(Client_number),
    FOREIGN KEY (Registration_number) REFERENCES
Vehicle(Registration_number),

    CHECK (Mileage_after IS NULL OR Mileage_after >= Mileage_before)
);
""")

# print("Hire_Agreement table created successfully.")
```

Hire_number	Date_started	Date_wish_end	Mileage_before	Mileage_after	Client_number	Registration_number
1	2025-01-15	2025-01-20	25000	25200	1	ABC123
2	2025-03-10	2025-03-15	32000	32500	2	DEF456
3	2025-06-05	2025-06-12	41000	41800	1	GHI789
4	2025-09-18	2025-09-25	50000	50750	3	JKL012
5	2024-12-01	2024-12-05	28000	28300	4	MNO345

This code first sets Hire_number as the primary key and lists the other following attributes. This code also sets two foreign keys (Client_number and Registration_number) and sets that the mileage after must be zero or greater.

Outlet Table

```
# Create Outlet table
cursor.execute("""
CREATE TABLE IF NOT EXISTS Outlet (
    Outlet_number INTEGER PRIMARY KEY,
```



```

Address TEXT NOT NULL,
Phone_number TEXT NOT NULL,
Fax_number TEXT

```

Outlet_number	Address	Phone_number	Fax_number
1	100 Miami Ave, Mia	305-111-1001	305-111-2001
2	200 Coral Way, Cora	305-222-1002	305-222-2002
3	300 Ocean Dr, Miam	305-333-1003	
4	400 Brickell Ave, Mi	305-444-1004	305-444-2004
5	500 Kendall Dr. Ken	305-555-1005	

This code creates the Outlet table by first setting Outlet_number as the primary key and includes all attributes. It also allows for null within the fax_number attribute.

Staff Table

```

# Create Staff table
cursor.execute("""
CREATE TABLE IF NOT EXISTS Staff (
    Staff_number INTEGER PRIMARY KEY,
    First_name TEXT NOT NULL,
    Last_name TEXT NOT NULL,
    Home_address TEXT NOT NULL,
    Home_phone_number TEXT NOT NULL,
    DOB TEXT NOT NULL,
    Sex TEXT NOT NULL CHECK (Sex IN ('M', 'F', 'Other')),
    Date_joined TEXT NOT NULL,
    Job_title TEXT NOT NULL,
    Salary REAL NOT NULL CHECK (Salary > 0),
    Outlet_number INTEGER NOT NULL,
    FOREIGN KEY (Outlet_number) REFERENCES Outlet(Outlet_number)
);
""")

```

Staff_number	First_name	Last_name	Home_address	Home_phone_num	DOB	Sex	Date_joined	Job_title	Salary	Outlet_number
1	Alice	Johnson	101 Staff Rd, Miami	305-201-0001	1980-03-14	F	2020-01-10	Manager	65000	1
2	Robert	Wilson	102 Staff Rd, Miami	305-202-0002	1979-08-20	M	2021-06-15	Sales Assistant	42000	1
3	Emily	Davis	103 Staff Rd, Miami	305-203-0003	1991-12-01	F	2022-03-22	Clerk	38000	2
4	James	Miller	104 Staff Rd, Miami	305-204-0004	1987-04-09	M	2019-09-01	Mechanic	50000	3
5	Olivia	Martinez	105 Staff Rd, Miami	305-205-0005	1993-10-17	Other	2023-02-11	Receptionist	36000	4

This code is setting staff_number as the primary key with all of its attributes. Sex is set to a non-integer value and Salary must be greater than 0. In addition, Outlet_Number is created as a foreign key.

Vehicle Table

```
# Create Vehicle table
cursor.execute("""
CREATE TABLE IF NOT EXISTS Vehicle (
    Registration_number TEXT PRIMARY KEY,
    Make TEXT NOT NULL,
    Model TEXT NOT NULL,
    Engine_size REAL NOT NULL CHECK (Engine_size > 0),
    Capacity INTEGER NOT NULL CHECK (Capacity > 0),
    Current_mileage INTEGER NOT NULL CHECK (Current_mileage >= 0),
    Daily_hire_rate REAL NOT NULL CHECK (Daily_hire_rate > 0),
    Outlet_number INTEGER NOT NULL,
    FOREIGN KEY (Outlet_number) REFERENCES Outlet(Outlet_number)
);
""")

#print("Vehicle table created successfully.")
```

Registration_number	Make	Model	Engine_size	Capacity	Current_mileage	Daily_hire_rate	Outlet_number
ABC123	Honda	Civic	1.8	5	25000	45	1
DEF456	Toyota	Camry	2.5	5	32000	55	1
GHI789	Honda	Accord	2	5	41000	60	2
JKL012	Ford	Explorer	3	7	50000	80	3
MNO345	BMW	X5	3	5	28000	120	4

This code creates the Vehicle table by first setting Registration_number as the primary key. Within the attributes, Engine_size, Capacity and Daily_hire_rate are set to be greater than zero. In addition, Current_mileage is set to greater than or equal to zero. Lastly, Outlet_number is set as the foreign key.

C. Queries

Query 1

Query 1: Vehicles hired by Client 1

(1, 'John', 'Smith', 'ABC123', 'Honda', 'Civic')

(1, 'John', 'Smith', 'GHI789', 'Honda', 'Accord')

```
print("\nQuery 1: Vehicles hired by Client 1")
```

```
cursor.execute("""
SELECT
    Client.Client_number,
    Client.First_name,
    Client.Last_name,
    Vehicle.Registration_number,
    Vehicle.Make,
    Vehicle.Model
FROM Client
JOIN Hire_Agreement
    ON Client.Client_number = Hire_Agreement.Client_number
JOIN Vehicle
    ON Hire_Agreement.Registration_number = Vehicle.Registration_number
WHERE Client.Client_number = 1;
""")

for row in cursor.fetchall():
    print(row)
```

This code joins three tables together: Client, Hire_Agreement and Vehicle. It is set to search through Client_number that matches with Vehicle.Registration_number for Client number 1.

Query 2

Query 2: Vehicles above 30,000 miles hired during 2025

('DEF456', 'Toyota', 'Camry', 32000, '2025-03-10', 'Maria', 'Garcia')

('GHI789', 'Honda', 'Accord', 41000, '2025-06-05', 'John', 'Smith')

('JKL012', 'Ford', 'Explorer', 50000, '2025-09-18', 'David', 'Lee')

```
print("\nQuery 2: Vehicles above 30,000 miles hired during 2025")
```

```
cursor.execute("""
SELECT
    Vehicle.Registration_number,
```

```

    Vehicle.Make,
    Vehicle.Model,
    Vehicle.Current_mileage,
    Hire_Agreement.Date_started,
    Client.First_name,
    Client.Last_name
FROM Vehicle
JOIN Hire_Agreement
    ON Vehicle.Registration_number = Hire_Agreement.Registration_number
JOIN Client
    ON Hire_Agreement.Client_number = Client.Client_number
WHERE Vehicle.Current_mileage > 30000
    AND Hire_Agreement.Date_started BETWEEN '2025-01-01' AND '2025-12-31';
""")

for row in cursor.fetchall():
    print(row)

```

This code combines data from tables Client, Hire_Agreement and Vehicle. It narrows down the scope to only vehicles that were hired between January 1st, 2025 and December 31, 2025, providing the result of all vehicles above 30,000 miles hired in 2025.

Query 3

Query 3: Total number of Honda vehicles across all outlets
('Honda', 2)

```

print("\nQuery 3: Total number of Honda vehicles across all outlets")

cursor.execute("""
SELECT
    Make,
    COUNT(*) AS Total_Hondas
FROM Vehicle
WHERE Make = 'Honda';
""")

for row in cursor.fetchall():
    print(row)

```

This code identifies the number Hondas in the Vehicle table in its entirety.

Query 4

Query 4: Vehicle make with the highest daily hire rate
(BMW, 120.0)

```
print("\nQuery 4: Vehicle make with the highest daily hire rate")
```

```
cursor.execute("""
SELECT Make, Daily_hire_rate
FROM Vehicle
WHERE Daily_hire_rate = (
    SELECT MAX(Daily_hire_rate)
    FROM Vehicle
);
""")
```

```
for row in cursor.fetchall():
    print(row)
```

This code is selecting Daily_hire_rate for all makes from the Vehicle table filtering down to the maximum Daily_hire_rate.

Query 5

Query 5: Top 10 outlets with the most staff

(1, 2)

(4, 1)

(3, 1)

(2, 1)

```
print("\nQuery 5: Top 10 outlets with the most staff")
```

```
cursor.execute("""
SELECT
    Outlet_number,
    COUNT(*) AS Staff_Count
FROM Staff
GROUP BY Outlet_number
ORDER BY Staff_Count DESC
LIMIT 10;
""")
```

```
for row in cursor.fetchall():
```

```
print(row)
```

This code searches through the Staff table using Outlet_number to provide a total amount of staff at each outlet using Staff_Count. Staff_count is then ranked from most to least (descending order).